Yes — this is **Week 3**, covering:

- **Session 5:** Optimality + unbalanced class problems
- **Session 6:** SVMs
- **Practical:** Unbalanced classes using Random Forest, threshold adjustment, reweighting, ROC curves and SMOTE

Below are your **exam-focused Week 3 notes** in the same two formats.

Week 3 — Optimality, Unbalanced Class Problems and SVMs

Part A — Structured & Formal Summary

1. Purpose of Week 3

Week 3 extends the model-comparison framework from Weeks 1–2. The main question is no longer only "Can the model use this data?" but also:

- What does the model optimise during training?
- Does the training algorithm find a global optimum, local optimum, or approximate solution?
- Does the model output probabilities?
- What happens when the output classes are imbalanced?
- How can imbalance be handled through thresholds, weights, resampling or different algorithms?
- How do Support Vector Machines work conceptually?
- When should linear, polynomial or RBF SVMs be used?

The lecture starts by adding **approximate versus optimal solutions** to the model comparison framework, then moves into **unbalanced output classes** and finally introduces **SVMs** as a new model family.

Session 5 — Optimality and Unbalanced Class Problems

2. What Does "Learning" Mean in Machine Learning?

All machine learning models have some component that is learned from data.

The lecture explains that models “learn” the best values of the parameters that define their learned component. “Best” means best according to some objective or optimisation criterion.

Examples:

Model	Learned component	What is being searched for?
Logistic regression	regression parameters	parameter values that maximise likelihood
Decision tree	split rules / if-then structure	splits that improve Gini impurity or information gain
SVM	hyperplane vector and decision point	boundary that maximises margin, with possible error penalty
Neural network	weights	weights that minimise loss

The key idea is:

A model links parameters to predictions, measures prediction quality on training data, then searches for parameter values that optimise the chosen criterion.

This is the general learning process for all models.

---

3. Objective Function versus Model Function

The lecture distinguishes between:

3.1 Model function

This is the function that produces predictions.

Example idea:

- input features go into the model,
- learned parameters are used,
- prediction comes out.

3.2 Objective function

This measures how good the model is on the training data.

For example, it may measure:

- classification accuracy,
- likelihood,
- sum of errors,
- Gini impurity,
- information gain,

- margin plus misclassification penalty.

The objective function is what the training algorithm tries to optimise. The lecture explicitly warns that the model function and the objective function are different.

---

4. Logistic Regression as an Example of Optimisation

For logistic regression, the learned parameters are regression parameters such as intercept and coefficient values.

The lecture explains that the best values can be understood intuitively as those that maximise classification accuracy, but for probabilistic models they are more formally learned by maximising a likelihood function.

So, logistic regression:

- uses a mathematical likelihood function,
- has an optimisation criterion: maximise likelihood,
- uses maximum likelihood as the optimisation method.

Exam point:

Logistic regression is not just "drawing a line." It learns parameters by optimising a probabilistic objective.

---

5. Decision Trees as an Example of Optimisation

For decision trees, the learned model is a set of if-then rules.

The objective is usually something like:

- reduce Gini impurity,
- maximise information gain,
- improve class separation at each split.

The algorithm is a custom greedy procedure that chooses splits step by step.

Important implication:

A decision tree does not usually search all possible trees globally. It greedily chooses locally useful splits, which can be fast and practical but may not produce the globally best possible tree.

---

6. Why Knowing the Objective Matters

You should know what a model is optimising because different objectives create different behaviour.

Example from the lecture:

- In regression, L2 loss penalises outliers more strongly than L1 loss.

This matters because two models can both seem reasonable but behave differently under noise, outliers, imbalance or different business costs.

Exam phrase:

Model selection should consider not only final score, but also whether the training objective matches the real task.

Optimality

7. Approximate, Global and Local Solutions

The search algorithm may or may not find the best solution.

The lecture separates three related ideas:

1. **Global optimum**
2. **Local optimum**
3. **Approximate solution**

These matter because a more complex model may be more expressive, but its training procedure may be less able to guarantee the best solution.

---

8. Globally Optimal Algorithms

A globally optimal algorithm learns parameters that truly optimise the objective function.

Meaning:

- if the objective function has a best possible solution,
- and the algorithm guarantees it can find that solution,
- then the result is globally optimal.

The lecture notes that globally optimal algorithms should be used if possible, but in practice they are usually associated with simpler, lower-complexity models.

Advantages:

- reliable optimisation,
- repeatable solution,
- easier to reason about,
- less risk that poor performance is caused by bad optimisation.

Disadvantages:

- may only be possible for simpler models,
- may not capture complex patterns.

---

9. Locally Optimal Algorithms

A locally optimal algorithm may find a solution that is best in its local region but not best overall.

The lecture explains this using the idea of an optimisation surface with peaks and valleys. Algorithms start somewhere, move iteratively according to their best guess, and can stop at a local maximum/minimum if the local direction says “stop.”

Advantages:

- widely used in practice,
- allows more complex models,
- often good enough.

Disadvantages:

- result may depend on starting point,
- may not be the best possible solution,
- random restarts or other tricks may be needed,
- more computationally expensive to improve reliability.

The lecture states that local optimisation is very widely used in practice.

---

10. Approximate Algorithms

Approximate algorithms optimise a simplified or approximate version of the objective.

This may happen because:

- the exact objective is too expensive,
- the dataset is too large,
- the model is too complex,
- a faster approximation is acceptable.

Approximate algorithms may still guarantee the global optimum of the approximation, or they may only find a local optimum of the approximation.

Trade-off:

- faster or more scalable,
- potentially lower classifier performance,
- may be necessary when exact training is impossible.

---

11. Why Optimality Matters in Model Choice

When choosing between algorithms, you must consider:

- Is the model expressive enough for the data?

- Can the algorithm find a reliable solution?
- Is the optimisation global, local or approximate?
- Is the additional model complexity worth the loss of optimisation guarantees?
- Are time and data-size constraints forcing approximation?

Exam-style answer:

A globally optimal simple model may be preferred when the relationship is simple and reliability matters. A locally optimal or approximate model may be preferred when the relationship is complex or data size makes exact optimisation impossible, but the analyst must accept the risk of suboptimal solutions.

Unbalanced Class Problems

12. Balanced versus Unbalanced Classes

In classification, many methods implicitly assume that output labels are roughly balanced.

Balanced classes means each class has a similar number of examples.

Examples that are often balanced or nearly balanced:

- customer type prediction,
- basic sentiment prediction,
- socio-demographic category prediction.

Unbalanced classes means one class has many more examples than another.

Examples from the lecture:

- advanced sentiment prediction,
- days until re-purchase,
- brick kiln detection in imagery,
- risk of child mortality at birth.

Extremely unbalanced classes include:

- fraud prediction,
- oil-split prediction,
- network intrusion detection.

---

13. Why Accuracy Fails with Imbalanced Classes

The first problem is evaluation.

If one class dominates, a model can get high accuracy by mostly predicting the majority class.

Example:

If 99% of cases are non-fraud, a model that always predicts “not fraud” gets 99% accuracy but detects no fraud.

The lecture states that accuracy can look too good, and changing the evaluation measure helps reveal whether the classifier is useful.

Important:

Changing the evaluation measure does **not** improve the model. It only tells you more honestly how useful the model is.

---

14. Why Algorithms Also Fail with Imbalanced Classes

The second problem is training.

Many algorithms internally optimise objectives that effectively favour the majority class because every point is weighted equally. If there are many more majority-class points, the model can improve its objective more by predicting the majority class well.

The lecture gives examples:

- SVM or logistic regression may place the boundary to favour the large class.
- Decision trees compute impurity using all points equally, so splits may favour the majority class.

Exam phrase:

In imbalanced data, the model may optimise the wrong objective for the business problem.

---

15. Recall and Specificity in Imbalanced Problems

For imbalanced binary classification, the lecture focuses on the trade-off between:

Recall

Accuracy for the positive / target / minority class.

Example:

- churners correctly identified,
- children at risk correctly identified,
- fraud cases correctly detected.

Specificity

Accuracy for the negative / majority class.

Example:

- non-churners correctly identified,

- children not at risk correctly identified,
- genuine transactions correctly identified.

The lecture states that in churn and risk-of-death examples, the analyst cares about the trade-off between per-class accuracy.

---

16. Balanced Accuracy

Balanced accuracy gives equal importance to each class.

Plain formula:

Balanced accuracy = average of recall and specificity

Use balanced accuracy when:

- classes are imbalanced,
- you do not have a custom business cost formula,
- you want equal weight per class.

The lecture says that if the default is uncertain, equal weights per class are often used, giving balanced accuracy.

---

17. The Goal in Imbalanced Classification

The goal is usually not simply “maximum accuracy.”

Instead, the goal is:

- improve minority-class accuracy,
- keep majority-class accuracy good enough,
- choose a trade-off that matches the business or social cost.

Examples:

- In churn prediction, missing churners loses customers, but wrongly targeting non-churners wastes discounts.
- In risk prediction at birth, missing risk can cost lives, but predicting too many people as high-risk may exceed available resources.

Methods for Handling Unbalanced Classes

18. Option 1 — Change the Probability Threshold

For probabilistic classifiers, the default threshold is often 0.5.

Example:

- predict positive if probability is greater than 0.5.

In imbalanced problems, you may lower the threshold for the minority class.

Example:

- predict churn if probability is greater than 0.25 instead of 0.5.

This increases minority-class predictions, usually increasing recall but reducing specificity. The lecture describes this as a post-hoc adjustment that does not require retraining the model.

Advantages:

- fast,
- easy to evaluate,
- no retraining needed,
- can explore recall-specificity trade-offs.

Disadvantages:

- does not change the learned boundary,
- may only approximate the desired solution,
- can cause procedural overfitting if the threshold is selected using the test set.

---

19. ROC Curve

A ROC curve helps examine threshold trade-offs.

It plots:

- true positive rate / recall,
- against false positive rate / fallout.

The practical labels the axes as:

- mistake percentage for bad wine / false positive rate,
- accuracy of predicting good wine / true positive rate.

The lecture notes that threshold changes allow you to re-evaluate metrics quickly and select a useful trade-off.

---

20. Option 2 — Cost-Sensitive Learning / Reweighting

Cost-sensitive learning changes the training objective so that errors on one class matter more.

Example from the lecture:

If there are 1000 times more non-churners than churners, and you want equal class accuracy, you can make each churner count 1000 times more than each non-churner.

This can be done by providing class weights or sample weights.

Advantages:

- changes the model training objective,
- can learn a better boundary for the desired trade-off,
- generalises naturally to multiple classes,
- closer to optimising the real objective than threshold adjustment.

Disadvantages:

- model must be retrained for each weighting,
- choosing weights may be difficult,
- extreme weights may overfit the minority class.

---

21. Threshold Adjustment versus Reweighting

The lecture gives an important recap:

Reweighting

- places relative importance on getting one class correct,
- retrains the classifier,
- learns a new boundary,
- can be closer to the optimal boundary for the desired costs,
- must retrain for different weights.

Probability threshold adjustment

- uses the model's existing probability estimates,
- changes how confident the model must be before predicting the positive class,
- can be evaluated without retraining,
- may approximate the effect of reweighting but does not learn the same boundary.

The lecture also states that both methods can be combined: reweight first, then tune the threshold.

---

22. Option 3 — Resampling

Resampling changes the training dataset so that classes are more balanced.

22.1 Oversampling

Duplicate minority-class examples.

22.2 Undersampling

Remove majority-class examples.

22.3 Cost-Proportionate Example Weighting

Sample examples in proportion to their cost or importance.

22.4 Synthetic sample generation

Generate new minority-class examples rather than simply duplicating existing ones.

The lecture gives examples:

- image transforms such as rotation or flipping,
- SMOTE.

SMOTE creates new minority-class instances by forming convex combinations of neighbouring minority-class examples.

Risks of SMOTE:

- synthetic points are not independent of original points,
- linked points may end up split between training and testing,
- this can create leakage if resampling is done before the split,
- minor overfitting may occur.

---

23. Option 4 — New Algorithms for Extreme Imbalance

When imbalance is extreme, thresholding, reweighting and resampling may not be enough.

Why?

The minority class may be so rare that it looks like noise.

The lecture introduces:

- anomaly detection,
- outlier detection,
- novelty detection.

---

24. Outlier and Anomaly Detection

Outlier/anomaly detection assumes the training data contains outliers that are far from most observations.

The goal is to fit the regions where training data is most concentrated while ignoring deviant observations.

In practice, outlier detection and anomaly detection are often similar.

Examples:

- fraud,
- network intrusions,
- abnormal customer behaviour,
- unusual sensor readings.

---

25. Novelty Detection

Novelty detection assumes the training data is **not polluted by outliers**.

The goal is to decide whether a new observation is unlike the normal training data. In this context, the outlier is called a novelty.

Difference:

- Outlier/anomaly detection: outliers may exist in training data.
- Novelty detection: training data is assumed clean; abnormality is detected in new data.

---

26. Types of Anomalies

The lecture lists three types:

26.1 Point anomalies

A single observation is abnormal.

Example: one fraudulent transaction.

26.2 Contextual anomalies

An observation is abnormal only in context.

Example: not attending a lecture might be unusual for one person but normal for another context.

26.3 Collective anomalies

A group pattern is abnormal even if individual points are not.

Example: everyone in a class going on holiday to the same location at the same time.

---

27. Density-Based Anomaly Detection

The lecture's basic assumption:

Normal data points occur around dense neighbourhoods; abnormalities are far away.

Methods:

27.1 k-Nearest Neighbour

Points far away from their k nearest neighbours are treated as noise or anomalies.

27.2 k-Means

Clusters are formed. Points far from cluster centres are treated as outliers. A key difficulty is choosing k.

27.3 DBSCAN

DBSCAN forms clusters by merging points within an eps distance if they have enough neighbours. Points that do not belong to a dense cluster are treated as noise.

Practical — Unbalanced Classes

28. Practical Task Overview

The practical revisits wine quality prediction but converts it into a binary classification task:

- `quality >= 7` becomes True / high-quality wine,
- all other wines become False / other-quality wine.

The practical uses:

- `train_test_split` with `stratify=y`,
- `RandomForestClassifier(max_depth=10, random_state=42)`,
- `KNNImputer`,
- sklearn `Pipeline`,
- accuracy,
- recall,
- specificity,
- balanced accuracy,
- ROC curve,
- sample weighting,
- SMOTE.

The practical explicitly warns that it uses a simple train-test split for teaching purposes and says this is not acceptable for real-world coursework evaluation.

---

29. Baseline Random Forest Result

The baseline Random Forest with default threshold 0.5 gives approximately:

Metric	Result
Accuracy	0.875
Recall	0.347

Metric	Result
Specificity	0.958
Balanced accuracy	0.653

Interpretation:

- Overall accuracy looks high.
- But recall is poor: only about 35% of high-quality wines are detected.
- Specificity is high: the model is good at identifying other-quality wines.
- This is a typical imbalanced-class pattern.

---

30. Threshold Adjustment in the Practical

The practical uses `predict_proba()` and `roc_curve()` to examine different thresholds.

Important procedure:

1. Use `.predict_proba(X_test)` instead of `.predict(X_test)`.
2. Extract the probability of the positive class.
3. Pass ground truth and probabilities into `roc_curve`.
4. Compare thresholds using recall, specificity and balanced accuracy.

The practical notes that choosing thresholds based on the test set risks **procedural overfitting**, because the model-selection process has used test-set knowledge.

---

31. Reweighting in the Practical

The practical reweights training instances so that minority-class examples count more.

Training set counts:

- class 0: 926
- class 1: 145

The upweight value is:

- 926 / 145 = about 6.386

This means each high-quality wine example is weighted around 6.386 times more than each other-quality wine example.

Weighted Random Forest at threshold 0.5 gives approximately:

Metric	Result
Accuracy	0.877
Recall	0.403

Metric	Result
Specificity	0.952
Balanced accuracy	0.677

Interpretation:

- Recall improves compared with the baseline.
- Specificity falls slightly.
- Balanced accuracy improves.
- Reweighting helps but does not completely solve the issue.

---

32. SMOTE in the Practical

SMOTE is then used to generate synthetic minority-class examples.

Before SMOTE:

- False: 926
- True: 145

After SMOTE:

- False: 926
- True: 926

The practical notes that SMOTE does not handle missing values directly, so KNN imputation is applied first.

SMOTE Random Forest at threshold 0.5 gives approximately:

Metric	Result
Accuracy	0.854
Recall	0.611
Specificity	0.893
Balanced accuracy	0.752

Interpretation:

- Accuracy falls slightly.
- Recall improves substantially.
- Specificity falls but remains reasonably high.
- Balanced accuracy improves clearly.
- This is a better trade-off if the goal is to detect more high-quality wines.

The practical's final table suggests that thresholding, reweighting and SMOTE can all help, and they can be combined, but the true improvement cannot be trusted without guarding against procedural overfitting.

Session 6 — Support Vector Machines

33. What Is an SVM?

A Support Vector Machine is a model that separates classes using a hyperplane.

In two dimensions, the hyperplane is a line.

In three dimensions, it is a plane.

In higher dimensions, it is called a hyperplane.

The lecture explains that data can exist in 2D, 3D, 4D or n-dimensional space depending on the number of features.

---

34. Hyperplane Concept

A hyperplane is defined by:

- a vector,
- and a point.

The vector can be understood as a discriminative feature: the fastest direction to move between the two classes. It is a linear combination of the input features.

Changing the point translates the decision boundary.

Changing the vector rotates the decision boundary.

Exam phrase:

An SVM learns a separating hyperplane by learning a vector and a decision point.

---

35. Prediction with a Linear SVM

For prediction, the SVM checks which side of the hyperplane a new point lies on.

Conceptually:

- one side is the positive class,
- the other side is the negative class,
- the decision boundary is where the point has no shared direction with the discriminative vector.

The lecture says computationally we do not need to explicitly learn or draw the boundary. We check the new point's relative location to the vector and point.

Linear SVM prediction complexity:

- O(d)

where d is the number of input features.

---

36. Which Hyperplane Should an SVM Use?

Many hyperplanes may separate the classes.

An SVM chooses the one that **maximises the margin**.

The margin is the uncertain region between the nearest points of each class. The SVM places the decision boundary in the centre of this margin.

This adds the assumption of **high class separability**.

Exam phrase:

A linear SVM chooses the separating hyperplane with the largest margin.

---

37. Hard-Margin SVM

A hard-margin SVM assumes that the data can be perfectly separated by a hyperplane.

Training objective:

- maximise the gap / margin.

Constraint:

- all training points must be classified correctly.

The lecture states that this becomes a constrained optimisation problem and is globally optimal under the separability assumption.

Training complexity shown:

- $O(nd^2)$

Prediction complexity:

- O(d)

where:

- n = number of data points,
- d = number of dimensions/features.

---

38. Soft-Margin SVM and Misclassification

Real data is often noisy or not perfectly linearly separable.

A soft-margin SVM relaxes the requirement that all points must be correctly classified.

It allows misclassifications or margin violations and balances:

- margin size,
- total error.

The lecture describes this as minimising inverse gap size plus total error.

---

39. The C Parameter

C is the SVM penalty parameter.

It controls the trade-off between:

- fitting the training data,
- allowing errors to get a wider margin.

Large C

- penalises misclassifications strongly,
- fits closer to the data,
- lower tolerance for error,
- higher risk of overfitting.

Small C

- tolerates more misclassifications,
- allows wider margin,
- trusts model assumptions more,
- useful when data is noisy,
- may underfit if too small.

The lecture states that absolute C values do not have meaningful interpretation; they must be selected empirically.

Exam phrase:

Low C means stronger regularisation; high C means weaker regularisation and stronger penalty for training errors.

---

40. Primal and Dual Formulations

SVMs can be trained in primal or dual form.

40.1 Primal

The primal formulation learns:

- the vector,
- the decision point,

directly in the feature space.

The lecture gives primal complexity as:

- $O(d^2n)$

40.2 Dual

The dual formulation learns weights per selected data point, especially support vectors.

The decision rule is based on similarity scores between points.

The lecture gives dual complexity as:

- $O(n^2d)$

In practice, the implementation may choose the correct option at runtime, and average-case $O(dn)$ is often observed.

---

41. Support Vectors

Support vectors are the data points on or near the margin edge.

They are the points that actually contribute to defining the decision boundary.

The lecture says only support vectors contribute to learning in the dual view.

Exam phrase:

Support vectors are the critical training points that determine the SVM boundary.

---

42. SVMs and High-Dimensional Measurements

The lecture states that SVMs draw hyperplanes to separate data points independently of the measured dimension. This means the curse of dimensionality is less of an issue when the data is measured in many dimensions but the true structure is not genuinely complex in all those dimensions.

Important distinction:

- High-dimensional measurement is not automatically bad for SVMs.
- A genuinely complex high-dimensional data structure can still be difficult.

Non-Linear SVMs and the Kernel Trick

43. Why Need Non-Linear SVMs?

A linear SVM can only draw a linear decision boundary.

If classes are not linearly separable in the original feature space, one option is to transform the data into a higher-dimensional space where a linear boundary becomes possible.

Example idea:

- original features: x and y,
- new feature: x^2 or y^2,
- after transformation, a line/hyperplane may separate the classes.

---

44. The Kernel Trick

Mapping data explicitly into a very high-dimensional space can be computationally expensive.

The kernel trick avoids explicitly computing the high-dimensional representation.

Instead, it computes similarity between points as if they had been mapped into that space.

The lecture states that the SVM decision rule and optimisation procedure can be written in terms of similarity functions between points. A kernel function calculates that similarity efficiently.

Exam phrase:

The kernel trick allows an SVM to learn non-linear decision boundaries by computing high-dimensional similarities without explicitly constructing the high-dimensional features.

---

45. Common Kernels

The lecture lists common kernels:

- linear,
- polynomial,
- RBF,
- sigmoid.

Different kernels are suited to different problems.

The best kernel depends on the data, domain knowledge and empirical performance.

---

46. Linear Kernel

Use a linear kernel when:

- a linear boundary is plausible,
- the number of features is large relative to the number of data points,
- data may already be linearly separable,

- computational efficiency matters.

The lecture recommends trying `sklearn.svm.LinearSVC` first when features are large relative to data points.

---

47. Polynomial Kernel

A polynomial kernel creates non-linear combinations of the original features.

It can model curved boundaries, but it usually has more parameters and can become computationally expensive.

Use when:

- domain knowledge suggests polynomial interactions,
- the boundary has polynomial-like structure,
- data size allows the computation.

Important meta-parameters may include:

- C,
- polynomial degree,
- kernel-specific coefficients depending on implementation.

---

48. RBF Kernel

The RBF kernel is often a good starting point when you need non-linearity.

The lecture states that RBF:

- has fewer parameters than polynomial,
- allows more non-linearity,
- includes the linear kernel as a special/limiting case through parameter choice,
- approximately includes sigmoid-like behaviour through parameter choice.

The lecture's practical guideline:

As a general rule, RBF kernels are a good start when you do not know which non-linear kernel to use.

---

49. RBF Kernel Intuition

The lecture gives an intuition:

- place a Gaussian-like bump over each data point,
- one class gets upright bumps,
- the other class gets upside-down bumps,

- points close together have overlapping effects,
- the learned hyperplane cuts through the transformed space.

This creates non-linear boundaries in the original space.

---

50. Gamma in RBF SVM

Gamma controls the width of the Gaussian-like influence around points.

High gamma

- narrow influence,
- points are less merged with neighbours,
- creates more “islands,”
- trusts data more,
- can overfit,
- may create almost one region per point.

Low gamma

- wider influence,
- smoother boundary,
- behaves more like a linear SVM,
- may underfit if too low.

The lecture states that gamma must be empirically learned.

Exam phrase:

High gamma increases model flexibility and overfitting risk; low gamma smooths the model and can make it behave more like a linear SVM.

---

51. RBF SVM Complexity

The lecture gives RBF SVM complexity as:

Training:

- $O(n^2d)$

Prediction:

- $O(nsv\ d)$

where:

- n = number of data points,
- d = number of features,

- nsv = number of support vectors.

This means RBF SVM can become expensive when there are many data points or many support vectors.

---

52. SVM Model Selection in Practice

In practice, you usually do not know:

- whether to use linear or RBF kernel,
- what value of C to use,
- what value of gamma to use for RBF.

These are meta-parameters and must be learned through model selection.

The lecture says to think of each combination of kernel and parameters as a new model. Testing "as you know it" breaks down when doing this, which leads into later material on proper model selection and avoiding overfitting.

---

53. Practical Guidelines for SVM Choice

From the lecture:

Use Linear SVM first when:

- number of features is large relative to data points,
- the data may already be linearly separable,
- extra computation from kernels is likely to hurt.

Use RBF SVM when:

- number of features is small or moderate,
- number of data points is large but still computationally manageable,
- non-linear separation is likely useful.

The lecture gives `LinearSVC` for linear SVM and `SVC` for RBF SVM.

Logistic Regression versus Linear SVM

54. Similarities

Both logistic regression and linear SVM:

- learn linear decision boundaries,
- use meta-parameters to control overfitting,
- can be regularised,

- can perform well on linearly separable or near-linearly separable data.

The lecture states that the differences are relatively small, but the error trade-offs differ.

---

55. Differences

Logistic regression

- optimises likelihood,
- naturally produces probabilities,
- with regularisation it trades uncertainty/probability error against model complexity,
- penalises points near the boundary because their predicted probability is uncertain.

Linear SVM

- maximises the margin,
- focuses on support vectors,
- does not naturally produce probabilities,
- uses C to trade off margin size against misclassification error,
- often a strong choice if you are willing to tune C.

The lecture concludes that, in general, if you are willing to set C, linear SVM is likely to be a better choice than logistic regression.

SVM Regression

56. SVM Regression Concept

SVM regression uses the idea of an epsilon tube.

Instead of separating classes, it tries to fit a centreline such that most points lie within an epsilon-width tube.

Errors are points outside the epsilon tube.

The model balances:

- tube fit,
- flatness/simplicity,
- tolerance of errors,
- penalty C for errors.

The lecture says that within kernel SVM, allowing more errors leads to flatter curves.

This is lower priority for your stated Week 3 outcomes, but it is useful as an extension.

Week 3 High-Yield Exam Checklist

You should be able to explain:

- what a model objective function is,
- why different objectives create different model behaviour,
- global optimum versus local optimum,
- why local optimisation may depend on starting point,
- when approximate algorithms are useful,
- why accuracy fails for imbalanced classes,
- difference between recall and specificity,
- why balanced accuracy is useful,
- threshold adjustment for probabilistic classifiers,
- ROC curves as threshold trade-off tools,
- cost-sensitive learning and sample weights,
- oversampling, undersampling and SMOTE,
- anomaly, outlier and novelty detection,
- density-based anomaly detection,
- SVM hyperplane, vector and decision point,
- margin maximisation,
- hard-margin versus soft-margin SVM,
- role of C,
- primal versus dual SVM,
- support vectors,
- kernel trick,
- linear, polynomial and RBF kernels,
- gamma in RBF,
- when to use LinearSVC versus SVC with RBF,
- why SVM model selection needs careful validation.

Part B — Q&A Summary

1. What is the main focus of Week 3?

Week 3 focuses on optimisation in machine learning models, unbalanced class problems, anomaly/novelty detection, and Support Vector Machines.

---

2. What does “learning” mean in a machine learning model?

It means finding the best values of the model's learned parameters according to an objective function.

---

3. What is an objective function?

An objective function measures how good a model's predictions are on training data or according to another optimisation criterion.

---

4. What is the difference between a model function and an objective function?

The model function produces predictions. The objective function measures how good those predictions are and is what the training algorithm tries to optimise.

---

5. What does logistic regression optimise?

It optimises a likelihood function, usually through maximum likelihood estimation.

---

6. What does a decision tree optimise?

It greedily chooses splits using criteria such as Gini impurity or information gain.

---

7. Why should we know what a model optimises?

Because the optimisation objective affects how the model behaves, especially with noise, outliers, imbalance and different business costs.

---

8. What is a globally optimal solution?

A globally optimal solution is the best possible solution for the objective function across the whole search space.

---

9. What is a locally optimal solution?

A locally optimal solution is the best solution in a nearby region, but it may not be the best overall solution.

---

10. Why can algorithms get stuck in local optima?

Because they move step by step from a starting point and may stop when no nearby move improves the objective, even if a better solution exists elsewhere.

---

11. What is an approximate algorithm?

An approximate algorithm optimises a simplified or approximate version of the objective, usually to reduce computation or handle large data.

---

12. When should globally optimal algorithms be preferred?

When the model is suitable for the problem and an exact reliable solution is feasible.

---

13. When might locally optimal or approximate algorithms be necessary?

When the model or dataset is too complex for exact global optimisation.

---

14. What is the main trade-off in optimality?

There is a trade-off between model complexity, computational feasibility and the ability to guarantee an optimal solution.

---

15. What is a balanced class problem?

A classification problem where each output class has roughly similar numbers of examples.

---

16. What is an unbalanced class problem?

A classification problem where one class has many more examples than another.

---

17. Give examples of extremely imbalanced problems.

Fraud prediction, network intrusion detection and rare event detection.

---

18. Why is accuracy misleading with imbalanced classes?

Because a model can achieve high accuracy by mostly predicting the majority class while failing to detect the minority class.

---

19. What is recall?

Recall is the accuracy for the positive or target class.

Plain formula:
Recall = TP / (TP + FN)

---

20. What is specificity?

Specificity is the accuracy for the negative class.

Plain formula:
Specificity = TN / (TN + FP)

---

21. What is balanced accuracy?

Balanced accuracy is the average of recall and specificity.

Plain formula:
Balanced accuracy = (recall + specificity) / 2

---

22. Why is balanced accuracy useful?

It gives equal importance to both classes, making it more suitable than accuracy for imbalanced data.

---

23. Why do algorithms themselves struggle with imbalance?

Because many algorithms weight each training point equally, so the majority class dominates the training objective.

---

24. What is the goal in imbalanced classification?

To improve minority-class performance while keeping majority-class performance acceptable.

---

25. What is threshold adjustment?

Changing the probability threshold needed to predict the positive class.

---

26. What happens if the positive threshold is lowered?

The model predicts more positives, usually increasing recall and reducing specificity.

---

27. Does threshold adjustment require retraining?

No. It is a post-hoc adjustment using the existing model's probability scores.

---

28. What is a ROC curve used for?

It shows how recall and false positive rate change as the classification threshold changes.

---

29. What is procedural overfitting?

Procedural overfitting happens when model decisions, such as threshold choice, are tuned using the test set.

---

30. What is cost-sensitive learning?

Cost-sensitive learning changes the training objective by giving higher weight to more important classes or errors.

---

31. How does reweighting help imbalance?

It makes errors on minority-class examples count more, encouraging the model to learn a boundary that predicts that class better.

---

32. What is the difference between reweighting and threshold adjustment?

Reweighting retrains the model and changes the learned boundary. Threshold adjustment changes predictions after training without changing the boundary.

---

33. What is oversampling?

Oversampling increases the number of minority-class examples, often by duplication.

---

34. What is undersampling?

Undersampling reduces the number of majority-class examples.

---

35. What is SMOTE?

SMOTE creates synthetic minority-class examples by combining neighbouring minority-class instances.

---

36. What is a risk of SMOTE?

Synthetic examples are linked to original examples, so improper splitting can cause leakage or overfitting.

---

37. When might anomaly detection be more suitable than normal classification?

When class imbalance is extreme and rare examples may be treated as noise by ordinary classifiers.

---

38. What is anomaly or outlier detection?

It detects observations that are far from the concentrated regions of the training data.

---

39. What is novelty detection?

It detects whether new observations are abnormal, assuming the training data is clean and not polluted by outliers.

---

40. What is a point anomaly?

A single abnormal observation, such as a fraudulent transaction.

---

41. What is a contextual anomaly?

An observation that is abnormal only in a particular context.

---

42. What is a collective anomaly?

A group of observations that is abnormal as a pattern.

---

43. What is the basic assumption of density-based anomaly detection?

Normal points occur in dense neighbourhoods, while abnormal points are far away or in low-density regions.

---

44. How can kNN be used for anomaly detection?

Points far away from their k nearest neighbours can be labelled as anomalies.

---

45. How can k-means be used for anomaly detection?

Points far from cluster centres can be labelled as outliers.

---

46. How can DBSCAN be used for anomaly detection?

DBSCAN treats points that do not belong to dense clusters as noise or anomalies.

---

47. What is an SVM?

An SVM is a model that separates classes using a hyperplane.

---

48. What defines a hyperplane?

A hyperplane is defined by a vector and a point.

---

49. What does the SVM vector represent conceptually?

It represents the discriminative direction, or the fastest direction to move between the two classes.

---

50. How does a linear SVM make predictions?

It checks which side of the hyperplane a new point lies on.

---

51. What is the prediction complexity of a linear SVM?

O(d), where d is the number of input features.

---

52. Which hyperplane does an SVM choose?

It chooses the hyperplane that maximises the margin.

---

53. What is the margin?

The margin is the gap or uncertainty region between the nearest points of the two classes.

---

54. What assumption does margin maximisation add?

It adds the assumption of high class separability.

---

55. What is a hard-margin SVM?

An SVM that assumes the data can be perfectly separated by a hyperplane.

---

56. What is a soft-margin SVM?

An SVM that allows some misclassifications or margin violations.

---

57. What does C control in an SVM?

C controls the penalty for errors relative to margin size.

---

58. What happens when C is large?

Misclassifications are penalised strongly, so the model fits closer to the training data and may overfit.

---

59. What happens when C is small?

More errors are tolerated, the margin can become wider, and the model is more regularised.

---

60. What does low C mean in terms of regularisation?

Low C means stronger regularisation.

---

61. When might you choose a low C?

When the data is noisy and you do not want the model to chase every training point.

---

62. What is the primal SVM formulation?

It learns the vector and decision point directly.

---

63. What is the dual SVM formulation?

It learns weights on selected training points and uses similarity scores for prediction.

---

64. What are support vectors?

Support vectors are the key points on or near the margin that determine the SVM boundary.

---

65. Why is the dual formulation important?

It enables the kernel trick and shows that the model can be expressed using similarities between points.

---

66. What is the kernel trick?

The kernel trick computes similarities as if data were mapped into a higher-dimensional space, without explicitly creating that space.

---

67. Why is the kernel trick useful?

It allows non-linear decision boundaries efficiently.

---

68. What are common SVM kernels?

Linear, polynomial, RBF and sigmoid.

---

69. When is a linear SVM appropriate?

When a linear boundary is plausible, the number of features is large relative to data points, or computational efficiency matters.

70. When is an RBF SVM appropriate?

When non-linear boundaries are likely useful and the dataset is computationally manageable.

---

71. Why is RBF often a good starting kernel?

It is flexible, has fewer parameters than polynomial kernels, and can model non-linear relationships.

---

72. What does gamma control in an RBF SVM?

Gamma controls how narrow or wide each point's influence is.

---

73. What happens when gamma is large?

The model becomes highly flexible, creates narrow local regions and may overfit.

---

74. What happens when gamma is small?

The model becomes smoother and can behave more like a linear SVM.

---

75. What is RBF SVM training complexity from the lecture?

$O(n^2d)$, where n is number of data points and d is number of features.

---

76. What is RBF SVM prediction complexity from the lecture?

O(nsv d), where nsv is the number of support vectors and d is the number of features.

---

77. What meta-parameters usually need tuning for SVMs?

Kernel choice, C and gamma for RBF SVM.

---

78. Why is SVM model selection difficult?

Because each combination of kernel and meta-parameters behaves like a different model, so careful validation is needed to avoid overfitting.

---

79. What is the difference between logistic regression and linear SVM?

Logistic regression optimises likelihood and naturally outputs probabilities. Linear SVM maximises margin and usually does not naturally output probabilities.

80. Which one did the lecture suggest may often be better if C is tuned?

The lecture suggests linear SVM is likely to be a better choice than logistic regression if you are willing to tune C.

---

81. How can SVMs output probabilities?

SVM scores or distances can be converted into probabilities using methods such as Platt scaling.

---

82. What is SVM regression?

SVM regression fits a centreline with an epsilon tube around it, penalising points outside the tube.

---

83. What is the epsilon tube?

A region around the regression line where errors are tolerated without penalty.

---

84. What does C control in SVM regression?

It controls the cost of points lying outside the epsilon tube.

---

85. What is the main exam message of Week 3?

The best model is not only the one with the highest score. You must understand what it optimises, whether optimisation is reliable, how class imbalance changes both evaluation and learning, and how SVMs use margins, regularisation and kernels to create linear or non-linear decision boundaries.